Harmony Search

Algoritmos Metaheurísticos · Búsqueda de armonías inspirada en la improvisación musical

Integrantes: Nieto Campos Ricardo
Mariscal Rodriguez Omar Jesus
Rodriguez Guijarro Roberto Emiliano
Gonzales Sandoval Ximena Montserrat



Antecedentes

- Creado por Zong Woo Geem en 2001
- Rompe con la tradicion de inspirarse y se basa en la creatividad
- Pasar de acordes discordantes iniciales a una composición óptima tras múltiples ensayos.



El Proceso de los Músicos

- **Músico / Instrumento:** Representa una *variable de decisión* en nuestro problema.
- **Rango del instrumento:** Representa el *rango de valores permitidos* para esa variable.
- **Armonía (El acorde en un instante):** Es el *vector solución* (una posible respuesta al problema).
- **Apreciación del público:** Es la *función objetivo* (Fitness). Evalúa qué tan "bien" suena la solución.



Las 3 Reglas de la Improvisación

Texto para la diapositiva (Viñetas): Durante la improvisación, un músico toma una nota basándose en tres posibles acciones:

- **Uso de la Memoria Armónica:** Tocar algo que ya conoce y sabe que suena bien.
- **Ajuste de Tono (Pitch Adjusting):** Tomar una nota buena de la memoria y modificarla ligeramente.
- **Aleatoriedad:** Tocar una nota completamente nueva al azar para sorprender.



Origen: la música como modelo

- Propuesto por Zong Woo Geem, Joong Hoon Kim y G. V. Loganathan en 2001.
- Metaheurística estocástica inspirada en cómo un grupo de músicos improvisa hasta lograr una armonía agradable.
- Guarda en una memoria las mejores soluciones y las recombina para crear otras nuevas.
- Sirve para optimización continua y discreta, sin necesidad de derivadas.

En la música	En el algoritmo
Músico	Variable de decisión
Nota (tono)	Valor de la variable
Armonía	Solución completa x
Estética	Función objetivo f(x)
Ensayo	Iteración
Memoria del músico	Memoria armónica (HM)

Los cinco parámetros

HMS	Tamaño de la memoria armónica: cuántas soluciones se guardan.	**≈ 10 – 50**
HMCR	Probabilidad de tomar un valor de la memoria (vs. uno aleatorio).	**0.7 – 0.95**
PAR	Probabilidad de ajustar finamente un valor tomado de la memoria.	**0.1 – 0.5**
bw	Ancho de banda: tamaño máximo del ajuste (fracción del rango).	**≈ 1 % del rango**
NI	Número de improvisaciones: criterio de paro del algoritmo.	**miles**

Idea central: cómo se improvisa cada nota

1. Memoria

prob. HMCR

Se toma el valor de esa variable de una armonía al azar guardada en la memoria.

2. Ajuste de tono

prob. HMCR × PAR

Al valor tomado de la memoria se le suma una pequeña variación de tamaño bw.

3. Aleatoria

prob. 1 − HMCR

Se inventa una nota nueva en todo el rango permitido. Aporta diversidad.

Ejemplo con HMCR = 0.9 y PAR = 0.3: de cada 100 notas…



Los pasos del algoritmo

1

Definir

Problema f(x), límites y parámetros.

2

Inicializar

Llenar la HM con HMS soluciones aleatorias y evaluarlas.

3

Improvisar

Crear una nueva armonía nota por nota (memoria, ajuste o azar).

4

Actualizar

Si es mejor que la peor de la HM, la reemplaza.

5

¿Paro?

Si se cumplieron NI iteraciones, devolver la mejor armonía.

Se repiten los pasos 3 y 4 hasta completar NI improvisaciones.

Ajuste de tono: `x_nueva = x_memoria + U(−1, 1) · bw`

Ejemplo paso a paso

Minimizar $f(x_1, x_2) = x_1^2 + x_2^2$ en $[-5, 5]$ · HMS = 4, HMCR = 0.9, PAR = 0.3, bw = 0.1

Memoria armónica inicial

#	x_1	x_2	f(x)
1	1.25	3.97	17.32
2	2.76	−2.75	15.18
3	−2.00	3.74	17.99
4	**−4.95**	**3.21**	**34.81 (peor)**

Improvisación de la nueva armonía

- x_1: $r = 0.80 < HMCR \rightarrow$ toma 1.25 de la armonía 1; $r = 0.30 \geq PAR \rightarrow$ sin ajuste.
- x_2: $r = 0.28 < HMCR \rightarrow$ toma −2.75 de la armonía 2; $r = 0.25 < PAR \rightarrow$ ajusta −0.11 ⇒ −2.86.

Nueva armonía (1.25, −2.86): f = 9.74
9.74 < 34.81, mejora a la peor, así que reemplaza a la armonía 4 en la memoria.

Implementación en Python

```
HM = rng.uniform(lower, upper, (hms, dim))    # 1. memoria
costos = np.array([f(h) for h in HM])

for _ in range(n_iter):
  nueva = np.empty(dim)
  for j in range(dim):
    if rng.random() < hmcr:   # 2. memoria
      nueva[j] = HM[rng.integers(hms), j]
      if rng.random() < par:   # 3. ajuste
        nueva[j] += rng.uniform(-1,1)*bw*rango
    else:   # 4. azar
      nueva[j] = rng.uniform(lower, upper)
  peor = np.argmax(costos)   # 5. actualizar
  if f(nueva) < costos[peor]:
    HM[peor], costos[peor] = nueva, f(nueva)
```

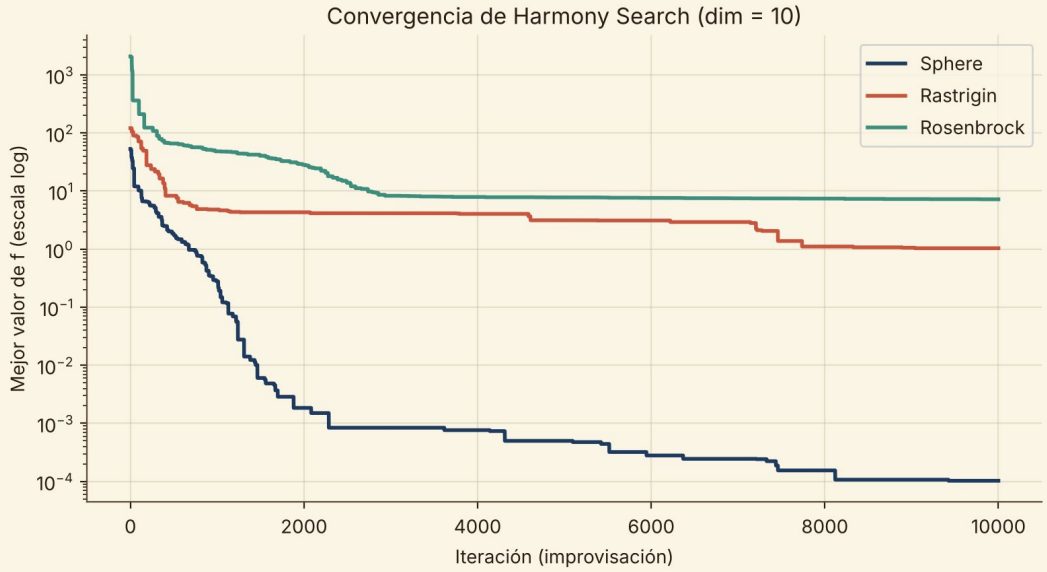
Archivo
harmony_search.py

Función
harmony_search(f, lower, upper, dim, hms, hmcr, par, bw, n_iter)

Pruebas
Sphere, Rastrigin y Rosenbrock

Ejecutar
python harmony_search.py

Resultados: convergencia

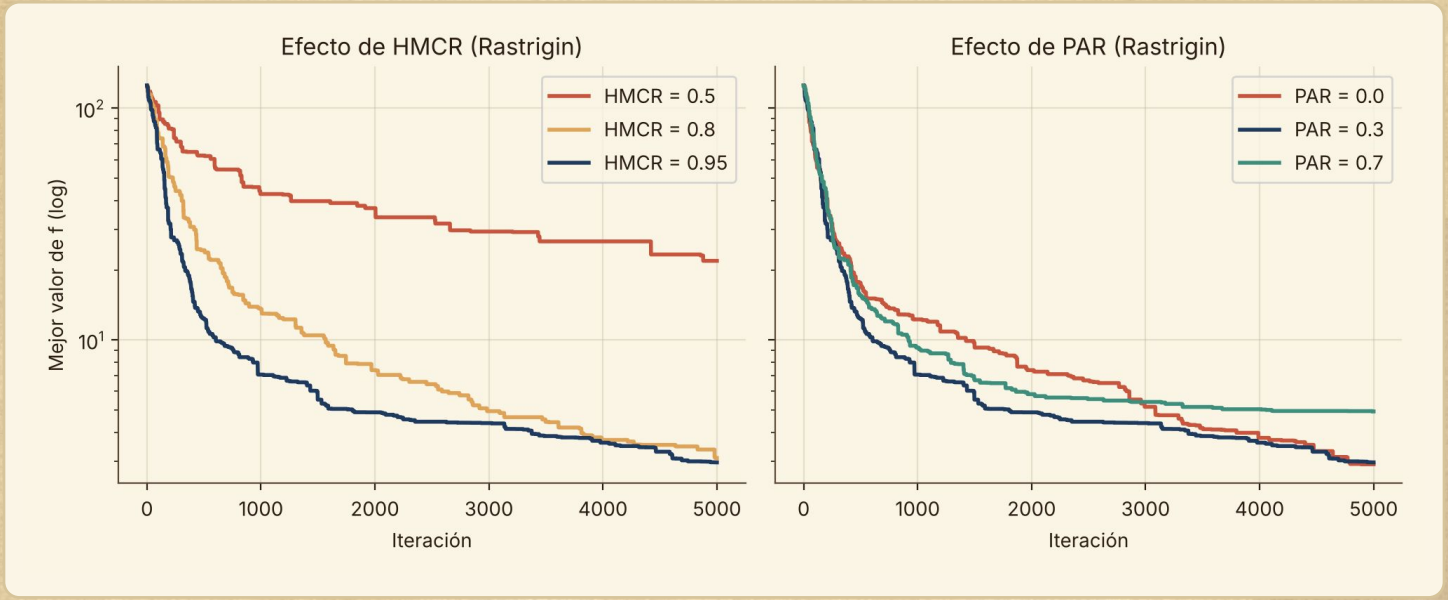


Función	Mejor f	Media f
Sphere	3.1e−5	7.5e−5
Rastrigin	5.4e−2	1.40
Rosenbrock	8.0e−2	4.74

20 corridas, dim = 10, 10 000 iteraciones, HMCR = 0.95, PAR = 0.3, bw = 0.01.

Excelente en funciones unimodales; en Rastrigin y Rosenbrock avanza más lento y tiende a estancarse.

Efecto de HMCR y PAR



- HMCR alto (0.95) converge mucho más rápido; con 0.5 casi es búsqueda aleatoria.
- PAR = 0.3 es un buen equilibrio; con 0.7 se estanca en esta prueba (5 corridas, 5 000 iteraciones).

Ventajas, límites y aplicaciones

Ventajas

- Simple y fácil de implementar
- Pocos parámetros
- No requiere derivadas
- Sirve en variables continuas y discretas

Limitaciones

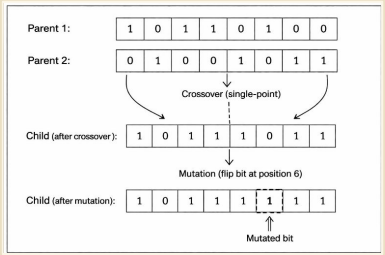
- Sensible a HMCR, PAR y bw
- Puede estancarse en problemas complejos
- Requiere afinar parámetros

Aplicaciones

- Diseño de redes de agua
- Diseño estructural
- Ruteo (agente viajero)
- Horarios y planificación

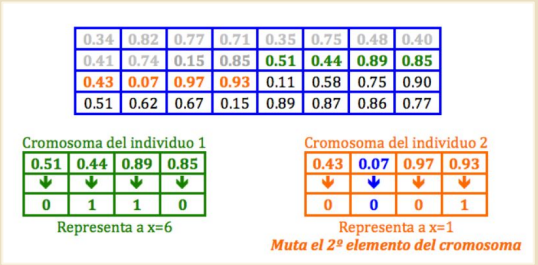
Comparación con otros algoritmos vistos en clase

Algoritmo Genético binario (AGB)



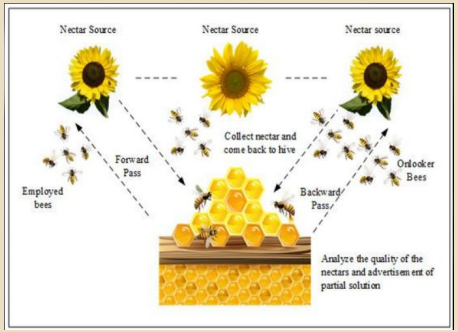
Similitud: Ambos utilizan soluciones anteriores para generar nuevas alternativas y mejorar los resultados.
Diferencia: El genético binario utiliza cadenas de ceros y unos, aplicando selección, cruzamiento y mutación. Harmony Search utiliza una memoria de soluciones, ajuste de tono y valores aleatorios.

Algoritmo Genético continuo (AGC)



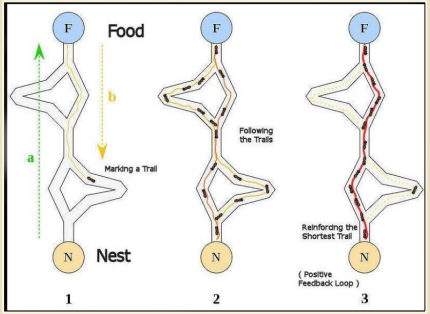
Similitud: Ambos trabajan con soluciones que pueden representarse mediante números reales.
Diferencia: El genético continuo utiliza cruzamiento y mutación para modificar sus soluciones, mientras que Harmony Search combina valores de su memoria y realiza pequeños ajustes.

Algoritmo de abejas artificiales (ABC)



Similitud: Ambos buscan mejorar soluciones existentes y explorar nuevas alternativas.
Diferencia: ABC imita el comportamiento de las abejas mediante abejas empleadas, observadoras y exploradoras. Harmony Search utiliza la memoria armónica para generar nuevas soluciones.

Algoritmo de optimización por colonia de hormigas (ACO)



Similitud: Ambos utilizan información de soluciones anteriores para orientar la búsqueda.
Diferencia: ACO utiliza feromonas para encontrar caminos favorables, mientras que Harmony Search utiliza una memoria de soluciones para crear nuevas alternativas.

Algoritmo de búsqueda gravitacional (GSA)

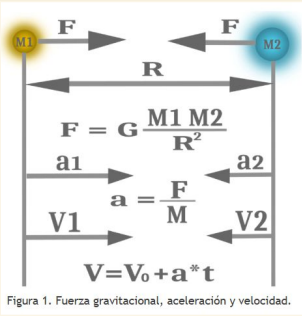
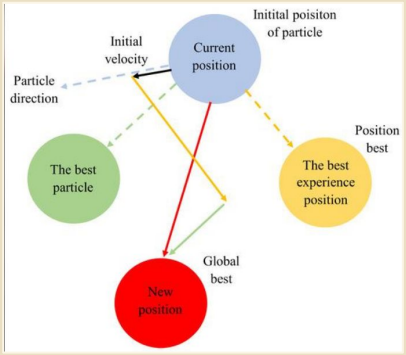


Figura 1. Fuerza gravitacional, aceleración y velocidad.

Similitud: Ambos mantienen un conjunto de soluciones candidatas e intentan mejorar su calidad.
Diferencia: GSA utiliza fuerzas gravitacionales para atraer a los agentes hacia mejores soluciones. Harmony Search genera soluciones mediante la selección y modificación de valores de su memoria.

Algoritmo de optimización de partículas (PSO)



Similitud: Ambos utilizan información de soluciones anteriores para orientar la búsqueda hacia mejores resultados.
Diferencia: PSO mueve partículas mediante velocidades y posiciones, tomando en cuenta la mejor solución individual y grupal. Harmony Search utiliza su memoria armónica para generar nuevas soluciones.

Tabla Comparativa general

Algoritmo	¿En qué se inspira?	¿Cómo encuentra soluciones?
Harmony search	Improvisación musical	Combina valores de la memoria, ajusta tonos y genera valores aleatorios.
Génetico binario	Evolución biológica	Selección, cruzamiento y mutación de cadenas binarias.
Génetico continuo	Evolución biológica	Selección, cruzamiento y mutación de valores reales.
Abejas artificiales (ABC)	Búsqueda de alimento de las abejas	Abejas empleadas, observadoras y exploradoras.
Colonia de hormigas (ACO)	Búsqueda de alimento de las hormigas	Construcción de caminos y actualización de feromonas.
Búsqueda Gravitacional (GSA)	Ley de gravedad de Newton	Atracción entre agentes según sus masas.
Partículas (PSO)	Movimiento colectivo de aves y peces	Actualización de velocidades y posiciones según mejores soluciones.

Conclusiones

- Harmony Search imita la improvisación musical: memoria, ajuste fino y azar.
- HMCR equilibra explotación y exploración; PAR y bw afinan la búsqueda.
- Es simple y eficaz, pero requiere calibrar sus parámetros.
- Harmony Search se caracteriza por su estructura sencilla, su flexibilidad y el uso de una memoria armónica para combinar y modificar soluciones existentes.

Referencias

Geem, Z. W., Kim, J. H. y Loganathan, G. V. (2001). A new heuristic optimization algorithm: Harmony Search. Simulation, 76(2), 60–68.

Lee, K. S. y Geem, Z. W. (2005). A new meta-heuristic algorithm for continuous engineering optimization: harmony search theory and practice. Computer Methods in Applied Mechanics and Engineering, 194(36–38), 3902–3933.